

Vue中`ref`的用法、原理及场景详解

你想让我详细讲解 Vue 中 `ref` 的用法、原理和使用场景，这是 Vue 3 中最基础也最核心的响应式 API 之一，我会从基础到进阶，把 `ref` 讲清楚。

一、`ref` 是什么？

`ref` 是 Vue 3 组合式 API 中用于创建**响应式数据**的核心函数，专门解决**基本数据类型**（数字、字符串、布尔值等）的响应式问题（因为 Vue 的 `reactive` 只能处理对象/数组的响应式）。

简单来说：`ref` 会把基本数据类型**包裹成一个带 `.value` 属性的响应式对象**，通过操作 `.value` 来修改数据，Vue 会自动追踪这个修改并更新视图。

二、`ref` 的基础用法

1. 基本使用步骤

Code block

```
1  <template>
2    <!-- 模板中使用时，不需要写 .value，Vue 会自动解包 -->
3    <div>计数：{{ count }}</div>
4    <button @click="addCount">加1</button>
5  </template>
6
7  <script setup>
8    // 1. 从 vue 中导入 ref
9    import { ref } from "vue";
10
11   // 2. 创建响应式数据：参数是初始值
12   const count = ref(0); // 包裹数字
13   const name = ref("张三"); // 包裹字符串
14   const isShow = ref(true); // 包裹布尔值
15
16   // 3. 方法中修改数据：必须通过 .value 操作
17   const addCount = () => {
18     count.value++; // 注意：JS 逻辑中必须写 .value
19     name.value = "李四"; // 修改字符串
20     isShow.value = !isShow.value; // 修改布尔值
21   };
22 </script>
```

2. 核心特点：.value 的使用规则

使用场景	是否需要 .value	原因
<template> 模板中	❌ 不需要	Vue 会自动为模板中的 ref 解包，直接用变量名即可
<script setup> 逻辑中	✅ 必须加	ref 本质是一个 包装对象 ，.value 才是真正存储数据的属性
reactive 中嵌套 ref	❌ 不需要（自动解包）	如果 ref 作为 reactive 对象的属性，访问时会自动解包（见进阶用法）

三、ref 的进阶用法

1. 引用 DOM 元素/组件（最常用的进阶场景）

ref 不仅能创建响应式数据，还能用来**获取 DOM 元素或子组件实例**，这是 Vue 中操作 DOM 的推荐方式。

Code block

```
1  <template>
2    <!-- 给 DOM 加 ref 属性，名称和脚本中的 ref 变量一致 -->
3    <div ref="box">我是一个 DIV</div>
4    <button @click="getDom">获取 DOM</button>
5  </template>
6
7  <script setup>
8    import { ref, onMounted } from "vue";
9
10   // 1. 创建一个空的 ref 用于接收 DOM
11   const box = ref(null);
12
13   // 2. 必须在组件挂载后（onMounted）才能获取 DOM
14   onMounted(() => {
15     console.log(box.value); // <div>我是一个 DIV</div>
16   });
17
18   // 3. 点击按钮获取 DOM
19   const getDom = () => {
20     box.value.style.color = "red"; // 修改 DOM 样式
21     console.log(box.value.innerText); // 输出：我是一个 DIV
```

```
22   };
23   </script>
```

2. ref 与 reactive 的配合使用

当 ref 作为 `reactive` 对象的属性时，Vue 会自动解包，访问时无需 `.value`：

Code block

```
1   import { ref, reactive } from "vue";
2
3   const age = ref(18);
4   const user = reactive({
5     name: "张三",
6     age: age, // 把 ref 嵌套到 reactive 中
7   });
8
9   console.log(user.age); // 18 (自动解包, 无需 .value)
10  user.age = 20; // 直接赋值, 等价于 age.value = 20
11  console.log(age.value); // 20 (数据同步)
```

3. ref 处理复杂数据类型

虽然 ref 主要用于基本类型，但也可以包裹对象/数组（底层会自动调用 `reactive`）：

Code block

```
1   const list = ref([1, 2, 3]);
2   list.value.push(4); // 修改数组, 视图会更新
3
4   const obj = ref({ a: 1 });
5   obj.value.a = 2; // 修改对象属性, 视图会更新
```

四、ref 的响应式原理（简单理解）

- `ref` 会创建一个 **RefImpl 实例**（包装对象），这个实例包含一个 `.value` 属性；
- 当你访问 `.value` 时，Vue 会收集依赖（比如模板中用到这个数据的地方）；
- 当你修改 `.value` 时，Vue 会触发依赖更新，重新渲染视图；
- 对于基本类型，`ref` 是直接监听 `.value` 的赋值；对于对象/数组，`ref` 内部会用 `reactive` 做深层响应式。

五、常见使用误区

1. 忘记加 `.value`：在脚本中修改 ref 数据时，必须加 `.value`，否则数据不会响应式更新：

Code block

```
1  const count = ref(0);
2  count++; // ❌ 错误：直接修改变量，不是修改 .value，视图不更新
3  count.value++; // ✅ 正确：修改 .value，视图更新
```

2. **提前获取 DOM**：在组件挂载前（比如直接在 `<script setup>` 根级别）获取 ref 对应的 DOM，结果会是 `null`：

Code block

```
1  const box = ref(null);
2  console.log(box.value); // null (组件还没挂载, DOM 不存在)
3  // 必须在 onMounted 钩子中获取
```

总结

1. `ref` 是 Vue 3 中创建**响应式数据**的核心 API，优先用于基本数据类型（数字、字符串、布尔值）；
2. 脚本中操作 ref 数据必须用 `.value`，模板中使用无需 `.value`；
3. `ref` 还有一个重要用途：获取 DOM 元素/子组件实例，需配合 `onMounted` 钩子使用。

（注：文档部分内容可能由 AI 生成）